



# Programmieraufgabe

## GreenBuy – Ein nachhaltiger Mini-Webshop



### Ziel der Aufgabe

Entwickeln Sie ein **sauber strukturiertes, wartbares und testbares Java-Backend** für einen nachhaltigen Webshop.

Alle zentralen Programmierkonzepte — **Collections, Streams, Dateioperationen, Unit-Tests, Mocks, Kapselung, saubere Architektur** — sollen angewendet werden.

---



### Fachliche Idee

GreenBuy ist ein nachhaltiger Webshop für **regionale Produkte** von Landwirten, Gärtnern und kleinen Produzenten. Ziel ist es, den Erzeugern **faire Preise** zu zahlen und gleichzeitig **möglichst geringe, aber realistische Verbraucherpreise** anzubieten. Die Produkte kommen mit **minimaler, umweltfreundlicher Verpackung** aus, und alle Lieferungen erfolgen **CO<sub>2</sub>-neutral** über lokale Lieferdienste.

Kund:innen haben eine Einkaufshistorie, die gespeicherte Produkte enthält. Sie können Produkte filtern und Statistiken einsehen.

Es gibt **keine grafische Oberfläche** – alle Funktionen werden per Konsole oder Tests ausgeführt.

---



### Anforderungen an das Datenmodell

#### 1. Produkt (Product)

Ein Produkt soll mindestens diese Eigenschaften besitzen, überlegt euch dafür die Datentypen:

| Feld                  | Beschreibung                            |
|-----------------------|-----------------------------------------|
| ID                    | eindeutige Produkt-ID (String oder int) |
| Name                  | Name des Produkts                       |
| Kategorie             | z. B. "Obst", "Gemüse", "Milchprodukte" |
| Preis                 | Verkaufspreis in Euro                   |
| CO <sub>2</sub> -Wert | positive oder negative Einsparung in kg |

## Technische Anforderungen

- Alle Felder sollen **private** sein.
  - Zugriff ausschließlich über **Getter** oder **immutable Konstruktion** (verschiedene Lösungswege möglich!).
  - equals() und hashCode() sinnvoll überschreiben  
→ Empfehlung: Nur über die ID
  - Optionale Bonusfrage: *Sollen Produkte immutable sein?*
- 

## 2. Kunde (Customer)

| Feld              | Beschreibung      |
|-------------------|-------------------|
| Kundennummer      | eindeutige ID     |
| Name              | Vor- und Nachname |
| Gekaufte Produkte | Einkaufshistorie  |

### Best Practice-Hinweis:

- Überlegt euch eine geeignete Datenstruktur für die Einkaufshistorie.
  - Die Einkaufshistorie darf **nur** über Methoden wie addProduct(Product product) verändert werden.
  - Bei der Rückgabe der Einkaufshistorie muss sichergestellt sein, dass sie von außen nicht modifizierbar ist.
- 

## Dateioperationen (File I/O)

### Einlesen

- Produkte werden aus einer Datei geladen (CSV oder JSON).
- Fehlerbehandlung:
  - ungültige Zeilen / fehlende Felder
  - Formatfehler

Dateioperationen sollen so gestaltet sein, dass sie gut testbar und mockbar sind. Verwendet dafür instanzbasierte Services statt statischer Methoden.

## Speichern

Mindestens eine dieser Daten soll exportiert werden:

- Statistiken (z. B. nach Kategorien, Durchschnittspreise, CO<sub>2</sub>-Gesamtwerte)
- Produktliste

### Best Practice-Hinweis:

→ Dateioperationen müssen **in einer eigenen Klasse** bleiben (Single Responsibility Principle).

---

## Filter & Streams

Implementieren Sie in einer **ProductService**-Klasse mindestens fünf typische Abfragen mit Streams:

1. **Alle Produkte unter einem bestimmten Preis**
2. **Alle Produkte einer Kategorie**
3. **Sortierter Produktkatalog**  
(z. B. nach Name, Preis oder CO<sub>2</sub>-Wert)
4. **Gesamte CO<sub>2</sub>-Einsparung eines Kunden**  
(Summe über alle gekauften Produkte)
5. **Gruppierung aller Produkte nach Kategorie**  
(→ `Collectors.groupingBy`)

### Hinweise

- Streams klar, sauber und gut lesbar implementieren.
  - Keine Logik in Datenklassen – alles in Services.
  - Methoden sinnvoll benennen.
- 

## Unit Tests (Pflicht)

### ProductService

- 100 % Testabdeckung
- Normale Fälle + Edge-Cases  
(leere Listen, ungültige Kategorien, etc.)

## FileService

- Mindestens das Lesen aus einer Datei soll mit Mocks getestet werden (z. B. simulierte Datei, simulierte Fehler)
- Test eines ungültigen Dateiimports (z. B. korruptes CSV)

**Hinweis:** Das Programmieren der Tests dauert genauso lange wie der eigentliche Code!

---

## Demo

Ihre Anwendung muss eine **vollständig ausführbare Demo über die main()-Methode** enthalten.

Diese Demo zeigt, dass alle Teile des Systems zusammenarbeiten.

**Die Demo muss folgende Schritte ausführen:**

### 1. Produktdaten einlesen

- Die Anwendung lädt zu Beginn die abgespeicherten Daten (Produkte oder Statistikdaten) aus einer CSV- oder JSON-Datei.
- Alle erfolgreich eingelesenen Daten werden in der Konsole ausgegeben.

### 2. Customer anlegen

- Es wird mindestens ein Customer erstellt.
- Einige der eingelesenen Produkte werden seiner Einkaufshistorie zugeordnet.

### 3. Ein weiteres Produkt hinzufügen

- Während der Demo wird **mindestens ein zusätzliches Produkt** der Einkaufshistorie hinzugefügt.
- Die aktualisierte Einkaufshistorie wird ausgegeben.

### 4. Streams demonstrieren

Alle implementierten Stream-Abfragen müssen sichtbar ausgeführt werden.

Dies kann entweder:

- direkt hintereinander **oder**
- über ein einfaches Konsolenmenü

geschehen.

## 5. Datenexport durchführen (Pflicht)

Zum Abschluss schreibt das Programm **entweder**:

- die gesamte Produktliste **oder**
- eine Statistik

in eine Datei.

---

### Basisfunktion (Mindestanforderungen)

Damit alle Funktionen demonstriert werden können, müssen folgende Basisdaten vorhanden sein:

- mindestens **5 Produkte** (eingelesen oder vorher erstellt)
  - mindestens **1 Customer**
  - Einkaufshistorie mit mindestens **2 gekauften Produkten**
- 

### Architekturhinweise (didaktische Empfehlungen)

Diese Vorgaben sind *Leitlinien*, keine unerbittlichen Regeln:

#### Kapselung

- Alle Klassenvariablen **private**.
- Listen niemals direkt zurückgeben → Kopien oder unmodifiable.

#### Single Responsibility

- Product: nur Daten
  - Customer: nur Daten
  - ProductService: Filter + Streams
  - FileService: Dateioperationen
-

## Präsentation & Bewertung (0–2 Punkte)

Sie dürfen:

- alle Hilfsmittel verwenden
- zusammenarbeiten
- KI nutzen
- Fragen stellen

### **ABER:**

Jede Person muss mir **ihre eigene Lösung vorstellen** und **fachliche Fragen dazu beantworten können**.

## **Bewertungsskala (0–2 Punkte)**

| <b>Punkte</b> | <b>Kriterien</b>                                                                                                                                                                                                                                                                                |
|---------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0 Punkte      | Keine oder kaum funktionierende Lösung UND/ODER offensichtliches Nichtverständnis der eigenen Umsetzung. Fragen können nicht beantwortet werden.                                                                                                                                                |
| 1 Punkt       | Grundlegendes Verständnis ist vorhanden, aber mit Unsicherheiten. Die Lösung funktioniert teilweise, enthält jedoch Fehler, Unsauberkeiten oder unvollständige Bereiche. Antworten sind teils korrekt, teils unsicher.                                                                          |
| 2 Punkte      | Die Lösung ist vollständig, korrekt und sauber umgesetzt. Die Person kann sie souverän erklären und alle fachlichen Fragen kompetent beantworten. Ein klares Verständnis der Konzepte ist erkennbar. Eine sinnvolle, vollständige und klar strukturierte Testabdeckung wird ebenfalls erwartet. |

Viel Erfolg! 